

Networked MPLPB

Growing a Knowledge Network from a Seed

Mitchell D. McPhetridge

Independent Researcher · September 2026 · revised with a reference implementation

Abstract

A durable knowledge network does not need a durable server. It can grow from self-describing local corpora: directories of plain HTML, joined by relative links, carrying enough of their own organization that a copy remains interpretable on another machine, to another reader, after the original author is gone. This paper calls such a corpus a *seed*, and describes how seeds become nodes, how nodes find each other through hubs that are themselves ordinary corpora, how questions are routed by declared scope rather than by global similarity, and how provenance survives the trip.

Five commitments separate this proposal from its neighbours. Routing decides where an answer should come from and never writes it. When several corpora plausibly own a question, the ambiguity is reported, not merged. Provenance is checkable, not merely asserted: every retrieved page reduces to a signature by the node that served it over a fingerprint that names the page's hash. Machine-derived material carries an origin depth, so it cannot re-enter the network looking human. And the rules a copy must satisfy travel inside the copy, in plain language, so that the validator is as recoverable as the corpus.

A standard-library-only Python reference implementation accompanies the paper. It implements seeds, growth, four replication relations, signed discovery descriptions, hubs, scope routing, provenance envelopes, file / archive / HTTP connectors, and twelve local-policy controls, each behaviourally tested. A mechanical run of the eight-condition Seed-to-Network experiment passes every metric on both file and HTTP transport. That result is reported with its limits: corpora, queries, and code share one author, and the tests that would actually falsify the architecture (blind stranger recovery, ablation against flat retrieval, adversarial peers, scale) are specified here but not yet run. A small scale probe already shows one limit: the reference hub's registration cost grows super-linearly with registry size.

Keywords: local-first knowledge, federated retrieval, provenance, digital preservation, content addressing, resource selection, epistemic laundering, retrieval-augmented generation

1. Introduction

1.1 The problem

Knowledge systems built for machine readers are usually built around a service: a vector database, an API, a hosted index, a model endpoint. The service is where the organization lives. When the service stops, what remains is a pile of files that no longer know what they are.

The MPLPB local web took the opposite position for a single machine. A corpus is an actual website: plain HTML pages in a directory, connected by relative links, starting at `index.html`, each page declaring its identity, scope, status, and supersession in metadata. A reader, human or model, arrives with no memory and recovers the corpus's organization from the corpus itself. The Smart Local front end showed that this is enough to route questions to the page whose declared scope owns them, to cite that page with full provenance, and to refuse to answer when nothing owns the question.

This paper asks what happens when there are many such corpora on many machines, maintained by people who do not coordinate, some of whom will stop maintaining anything. The answer proposed is that the network should be made of the same material as its parts. A corpus that can be copied and still interpreted is a *seed*. A machine that carries seeds and says so is a *node*. A list of node descriptions, kept as an ordinary corpus, is a *hub*. Nothing in that stack has to stay running for the rest to stay meaningful.

1.2 What is new here

Almost every individual mechanism in this paper already exists somewhere. Replication for preservation is LOCKSS. Content-addressed fingerprints are Merkle trees and Git. Signed append-only identity is Secure Scuttlebutt. Routing a query to the collection most likely to hold the answer is resource selection in federated search. Describing a dataset so a stranger can use it is OAIS representation information and the VoID vocabulary. Offline self-contained corpora are Kiwix. Section 2 treats each of these seriously.

The contribution is not a new primitive. It is a specific combination, aimed at a reader that did not exist when those systems were designed: a stateless machine reader that arrives with no memory, reads whatever it is given, and will confidently merge, paraphrase, and forget where things came from unless the structure it reads prevents it. For that reader, five properties matter together, and no system surveyed provides all five:

- 1. Routing is separated from answering, and ambiguity is a first-class outcome.**
Federated search selects collections in order to merge their results. Here, merging across declared scopes is prohibited. When two corpora plausibly own a question, the correct output is a report of the collision with each corpus's lineage, not a blended answer. `ambiguous` sits beside `local`, `remote`, `not_found`, and `unavailable` as a normal result.
- 2. Epistemic origin travels as network metadata, and serving policy stays local.**
Every page declares whether it is human-authored, machine-derived, or ratified, and how many derivation steps separate it from a human source. Each node decides the maximum depth it will serve. Ratification, the one operation that resets depth, must itself be recorded. This targets a failure specific to machine readers: model output

re-entering the corpus as if it were source.

3. **The discovery layer has the same persistence conditions as the content.** A hub is not a service. It is a corpus whose pages are signed node descriptions, so it validates, copies, mirrors, and survives by exactly the mechanism it indexes, and can be rebuilt from descriptions surviving on any node.
4. **The validator is part of what is preserved.** Every seed carries its structural rules in plain prose, generated from the same table the code enforces, so a reader decades later can re-implement the checks without the software that first ran them. Most preservation formats specify their rules in an external standard; here the corpus carries them.
5. **Infrastructure burden is the dependent variable.** The architecture is judged by what can be removed while it still works: the origin node, the hub, the network, the author. The experiment is built as a sequence of removals, and the reference implementation uses nothing outside the Python standard library, including its signature scheme.

A skeptical reader will ask why this is not simply *a Git repository containing a static site and a manifest*. It nearly is, and that is intended; Section 2.8 answers the question directly. The short version: Git preserves bytes and history but says nothing about which repository owns which questions, whether two repositories collide, whether a file was machine-written, or how a reader should behave when the owner is unreachable. Those are the parts that matter to a machine reader, and they are the parts specified here.

1.3 What this paper does not claim

Structural validity is not truth. A corpus that passes every check can be wrong on every page. A signature proves which machine served a page, not that the page is correct. Origin depth measures derivational distance, not reliability. Declared scope is a claim by the author, and the router is only as good as the declarations. None of the results reported here involve a participant who has not seen the corpus, and none involve more than one machine or more than 160 nodes.

1.4 Contributions and organization

This paper contributes: (i) a definition of a seed as a six-part self-describing corpus; (ii) a node and hub model in which discovery is itself a corpus; (iii) four declared replication relations with hash-chained lineage; (iv) scope routing with a vocabulary, an ownership margin, and ambiguity preservation; (v) a verifiable provenance envelope; (vi) scoped supersession across forks; (vii) origin-depth rules including ratification records; (viii) a standard-library reference implementation with sixteen structural checks and twelve policy controls; (ix) an executable Seed-to-Network experiment with a report of what it does and does not show; and (x) a blind-recovery protocol and ablation design for the tests that remain.

Section 2 places the work in its field. Sections 3 through 13 specify the architecture. Section 14 describes the implementation, Section 15 the experiment and its results, Section 16 what those results do not establish. Sections 17 to 19 give failure modes, falsifiers, and open problems.

2. The Field

This section is organized by the question each body of work answers, and for each it states what MPLPB takes from it and where it departs. Citations are given from memory and should be checked against the sources before publication (see References).

2.1 Replication as preservation: LOCKSS

LOCKSS ("Lots Of Copies Keep Stuff Safe") preserves content by having many independent libraries each hold a copy and periodically audit one another by polling on content hashes, repairing damaged copies from peers. The core insight, that survival comes from independent copies rather than from a durable master, is exactly the premise of the seed. MPLPB departs in two ways. LOCKSS copies are meant to stay identical; seeds are meant to diverge, so the relation between copies (mirror, descendant, fork, partial) must be declared rather than repaired toward consensus. And LOCKSS preserves content for human librarians; MPLPB is concerned with what a machine reader does with the content, which is where routing, ambiguity, and origin depth come from.

2.2 Understandability over time: OAIS

The Open Archival Information System reference model (ISO 14721) distinguishes the content itself from the *representation information* needed to interpret it, and defines a *designated community* whose knowledge base the archive may assume. The death test in this paper is, in OAIS terms, a demand that the representation information travel inside the package and that the designated community be "a stranger with a browser and a hash tool". The seed's spec/ pages, boot block, and prose validation rules are representation information. What OAIS does not specify, because it is a model of an archive rather than of a network, is how independent archives route questions among themselves or how derived content should be labelled.

2.3 Content addressing and signed logs: Merkle, Git, IPFS, Secure Scuttlebutt, Hypercore

Hash-based integrity descends from Merkle's tree construction. Git applies it to history: every commit names its parent by hash, so tampering is detectable and forks are cheap. IPFS names content by hash so that any holder can serve it and the name still verifies. Secure Scuttlebutt gives every participant a key pair and an append-only signed feed, so identity is cryptographic and replication is gossip. Hypercore (Dat) does similar work with signed append-only logs.

MPLPB uses all of these ideas in their plainest form: a per-corpus manifest of SHA-256 hashes whose own hash is the corpus fingerprint; a hash-chained lineage file; node identity derived from an Ed25519 public key. What it adds is meaning on top of integrity. A content hash says the bytes are unchanged; it does not say which questions those bytes are authoritative for, whether they were machine-written, or whether a fork's revision should be read as the origin's. Those are declarations, and the network behaves according to them.

2.4 Resource selection in federated search: GLOSS, CORI, STARTS

Distributed information retrieval studied the problem of choosing which of many text collections to send a query to, using summaries of each collection's vocabulary. GLOSS estimated how many relevant documents each source held; CORI adapted inference networks to rank collections; protocols such as STARTS proposed standard source descriptions. This is the closest prior work to MPLPB routing, and MPLPB's discovery descriptions are recognizably source descriptions.

Two differences are deliberate. First, federated search estimates relevance from collection statistics; MPLPB routes by *declared* scope, a claim the author makes and a reader can inspect. This is cruder, and Section 7 is candid about what it costs. Second, federated search exists to merge results into one ranked list. MPLPB forbids that merge when ownership is unclear. For a human scanning a result list, merging is a convenience. For a machine reader that will write a paragraph from whatever it receives, merging two domains is the false-merger failure.

2.5 Harvesting and describing collections: OAI-PMH, VoID, webrings and directories

OAI-PMH lets repositories expose metadata for harvesting by aggregators, with low barriers to participation. VoID describes linked datasets so consumers can discover them. Webrings and early web directories were human-curated lists of sites, and a directory page was itself just a page. MPLPB hubs sit in this tradition: a hub harvests descriptions and publishes a list. The difference is that a hub is required to be an ordinary corpus of the same format as what it lists, so that losing a hub loses nothing a copy of the hub cannot restore, and each listed description is signed by the node it describes, so the hub cannot rewrite what it lists.

2.6 Provenance vocabularies and packaging: W3C PROV, nanopublications, BagIt, RO-Crate

W3C PROV models entities, activities, and agents, and the derivations among them. Nanopublications attach provenance to individual assertions and, with trusty URIs, make them content-addressed. BagIt packages files with a manifest of checksums; RO-Crate packages research objects with JSON-LD metadata. MPLPB's corpus manifest is essentially a bag manifest, and its derived-from and ratified-by fields are a very small subset of what PROV can express. The contribution here is not a richer vocabulary. It is a single integer, origin depth, chosen because a serving policy can act on it, and a rule that the only operation that lowers it must leave a record.

2.7 Offline and archival web: Kiwix / ZIM, Memento

Kiwix distributes whole reference corpora, such as Wikipedia, as single ZIM files readable without a network. Memento adds time to the web by linking resources to their prior states. Seeds share Kiwix's assumption that offline is normal, and share Memento's refusal to let the present version erase the past (retired pages remain in `_log/superseded/`). They differ in being written to be grown and forked by their holders, not only read.

2.8 Machine readers: retrieval-augmented generation and tool protocols

Retrieval-augmented generation conditions a model's output on retrieved passages. Tool protocols such as the Model Context Protocol give models structured access to external systems. Both are about how a model reaches knowledge; neither specifies how that knowledge is organized, owned, versioned, or labelled at the source, and both typically assume a running service. MPLPB is the other half: a knowledge format that a RAG pipeline or a tool server could sit on top of, which carries its own boundaries so the model does not have to invent them.

Why not a Git repository with a static site and a manifest? It is a fair baseline, and a seed can be stored in one. Git contributes integrity, history, and cheap forks. It does not contribute: a declared scope that routes questions; a rule that ambiguous ownership is reported; a distinction between a fork continuing a domain and a descendant starting a new one; per-page origin and depth; a discovery layer that is itself a repository of the same kind; or behaviour for an unreachable owner. The reference implementation could replace its manifest and lineage with Git objects without changing any of those properties. That is a sign the contribution lies in them.

2.9 Summary

Property	LOCKSS	OAIS	Git	IPFS / SSB	Fed. search	Kiwix	MPLPB
Survives loss of origin	yes	yes	yes	yes	no	yes	yes
Integrity by hash	yes	per package	yes	yes	no	partial	yes
Divergent copies with declared relation	no	no	partial	partial	no	no	yes
Routing by declared scope	no	no	no	no	estimated	no	yes
Ambiguity reported, not merged	no	no	no	no	no	no	yes
Origin depth / laundering control	no	no	no	no	no	no	yes
Discovery layer is itself preserved content	partial	no	no	partial	no	no	yes
Validation rules carried in the package	no	external	no	no	no	no	yes
Works with zero running services	no	n/a	yes	partial	no	yes	yes

The table compresses a great deal and is meant to show where the gaps are, not to score systems against goals they never had.

3. From Local Corpus to Seed

A local MPLPB corpus is already close to portable. Its pages use relative links, its root is `index.html`, and its metadata is in the page. What it lacks is an answer to three questions a stranger will ask of a copy: *which corpus is this*, *is this copy intact*, and *what rules must it obey*.

A **seed** is a bounded corpus that answers them. Formally,

$$S = (A, I, R, P, V, B)$$

where *A* is the set of artifacts (the HTML pages), *I* the identifiers (document IDs and one corpus ID), *R* the relationships (relative links, supersession, lineage), *P* provenance (owner, origin, origin depth, revision log), *V* validation (the rules, in prose and in code), and *B* the boot artifact (the Main Index boot block and its `B00T.md` twin).

On disk, the network-facing parts live in one directory, `_net/`, as plain JSON with an HTML twin a browser can read:

File	Contents
<code>_net/corpus.json</code>	corpus ID, title, kind, declared scope, scope terms, relation, seeded-from
<code>_net/vocabulary.json</code>	scope terms and their synonyms
<code>_net/lineage.json</code>	append-only, hash-chained replication history
<code>_net/catalog.json</code>	derived per-document records, for static retrieval
<code>_net/manifest.json</code>	SHA-256 of every file; its hash is the corpus fingerprint
<code>_net/_index.html</code>	human-readable twin, carrying the hash of <code>corpus.json</code>

The twin rule is inherited from the local web: when a machine file and its human twin could disagree, validation checks that they do not. Here the twin carries the SHA-256 of `corpus.json`, so a hand edit to one without the other fails check N.1.

A seed also carries `spec/seed.html`, which tells a reader holding only the directory how to open it, verify it with any SHA-256 tool, decide what kind of copy it is, and make it a node; and `spec/validation.html`, which states every structural rule in plain language.

The growth test. A seed is well-formed when a copy of it, on another machine, can produce another well-formed copy without contacting anyone:

$$\text{Valid}(S) \text{ and } \text{Copy}(S, M1 \rightarrow M2) \Rightarrow \text{Valid}(S') \text{ and } \text{CanSeed}(S')$$

What does not live in a seed is as important as what does. Node identity, keys, endpoints, peers, and local policy belong to the machine carrying the corpus. If they lived in the corpus, copying the corpus would copy an identity, and two machines would claim to be the same node (FM-N14). Check N.5 rejects any key or node file inside a corpus.

4. Nodes and Discovery Descriptions

A **node** is a machine that carries one or more corpora and says so:

$$N_i = (ID_i, S_i, D_i)$$

ID_i is its identity, S_i the set of corpora it carries, and D_i its **discovery description**, a small document that answers seven questions without requiring the node to reason: who are you; what corpora do you carry; what scope does each declare; where is each boot artifact; how can each be retrieved; when was this description last updated; and which other nodes do you know.

Identity is a key, not a name. Each node holds an Ed25519 key pair, and its ID is N- followed by the first sixteen hex digits of the SHA-256 of its public key. The description is signed with the corresponding secret. A reader accepts a description only if the signature verifies and the ID matches the key. Two machines therefore cannot claim the same ID without one failing verification. Identity collision stops being an overwrite and becomes a detectable forgery, recorded in known/collisions.json.

The standard library has hashing but no public-key signing. Rather than accept an unverifiable provenance chain or a dependency, the reference implementation includes RFC 8032 Ed25519 in about 150 lines of pure Python, checked against the RFC's test vectors. It is not constant-time and has not been audited; it provides tamper evidence, not protection against an adversary who can time the CPU. The wire format is standard, so a vetted library can be substituted without changing anything else. This is a real tension with the cheap-node hypothesis, and it is resolved here in favour of verifiability at some cost in speed.

A node's directory is:

node.json	signed discovery description
node.html	readable twin
config.json	local policy
local/	key, name, held-as records; never served, never packed
corpora/<corpus-id>/	the corpora carried
known/<node-id>.json	descriptions learned from peers and hubs
cache/	remote pages retrieved, with envelopes

Retrieval is static. A node need only serve files; the reader does the rest (Section 8). An old machine running any static web server, a shared folder, or a USB stick in a drawer is a complete node.

5. Replication: Mirror, Descendant, Fork, Partial

Replication is not synchronization. After a copy, the two directories may diverge, and the relation between them has to be declared rather than inferred from how similar they look.

Relation	Corpus ID	Changes allowed	Meaning
mirror	same	none	this is the same corpus, byte for byte
descendant	new	any	a new corpus seeded from the source, owning a new domain
fork	new	any	a new corpus continuing the source's domain under separate governance

Relation	Corpus ID	Changes allowed	Meaning
partial	new	only spokes removed	a new corpus carrying some of the source's spokes

A **mirror** must reproduce the source fingerprint exactly; otherwise it is not a mirror. Which machines hold a mirror is recorded by those machines, not inside the corpus, so the mirror stays byte-identical.

Every other relation mints a new corpus ID and records `seeded_from`, naming the source's corpus ID and fingerprint at the moment of copying, then appends a lineage event whose `prev` field is the SHA-256 of the previous event. The chain can be verified by anyone and cannot be edited without detection.

Lineage is not endorsement. A descendant that records "seeded from C-B64CA3EF6BDC at fingerprint cccd1db3..." says where it came from. It does not say the source has reviewed anything written since. Readers and routers must treat those as different claims, and the corpus identity page says so on its face.

The source must verify before it can be copied. The reference implementation refuses to replicate a corpus whose manifest, identity, or lineage fails, so corruption does not propagate by replication (FM-N12).

6. Hubs as Corpora of Corpus Descriptions

A **hub** is a node whose one special corpus lists other nodes:

$$H = \{ D_1, D_2, \dots, D_n \}$$

Each registered node gets a page `nodes/<node-id>.html` containing its signed description verbatim, and `nodes/registry.json` lists them with a `last_seen` time. Because the hub corpus is an ordinary MPLPB corpus, it validates under the same sixteen checks, copies by the same four relations, and is mirrored by the same mechanism as everything it indexes.

A hub answers one question: *where should I look?* It does not store the corpora it lists, answer their questions, rank them by quality, summarize them, or reconcile them. The reference implementation has no function in the hub module that takes a query and returns content. A hub that grew one would be hub capture (FM-N4), and it would show up in the diff.

A hub cannot forge a description; each is signed by the node it describes and verified one by one when a node learns from the hub. A hub can still omit descriptions or keep dead ones. Omission is visible only by comparing hubs, which is an argument for more than one. Dead entries are handled by `last_seen` and staleness (FM-N5). Re-registering a node supersedes its page rather than overwriting it, so a hub's own history is recoverable.

Hubs can list hubs. A node learning from a hub follows hub-to-hub links up to a configured number of hops.

Rebuilding a hub. Because descriptions are signed, a hub can be rebuilt from descriptions that survive anywhere: in other nodes' known/ directories, in caches, in a mirror of the old hub. A rebuilt hub is exactly as trustworthy as the original, since trust never rested on the

hub.

7. Routing by Declared Scope

Scope is the routing primitive. Every corpus declares a scope sentence and a list of scope terms, and every scope term has an entry in the corpus vocabulary with its synonyms. A router scores each known corpus against a query:

- two points for each distinct query term that matches a declared scope term or one of its synonyms;
- one point for each distinct query term that appears in the scope sentence.

Let $s_1 \geq s_2$ be the two highest scores. The router's outcome is:

Outcome	Condition
local	$s_1 \geq \text{min_score}$, $s_2 \times \text{margin} \leq s_1$, and this node carries the owner
remote	the same, and another node carries the owner
ambiguous	two or more corpora score within the margin of the leader, and local precedence does not resolve it
not_found	no corpus reaches min_score

A fifth outcome, unavailable, belongs to retrieval: the owner is known and cannot be reached.

This is declared ownership, not semantic similarity, and it is only as good as the declarations. The review of the first draft of this paper made the point sharply: deciding whether a query falls inside a scope is itself a semantic judgment, so the router's competence decides routing accuracy. If scopes are free text written by independent authors, routing accuracy measures the reader, not the architecture. The response here has three parts.

First, scope terms come from a **vocabulary carried in the seed**. Descendants inherit it and extend it. Corpora grown from the same seed therefore share a minimal taxonomy by construction, and a query phrased in a synonym ("bike", "furnace") routes as if it used the declared term. This does not solve vocabulary mismatch between unrelated seeds; it makes the mismatch a property of lineage that can be measured.

Second, the router is **inspectable**. `mplpb-net route` prints each scoring candidate, its score, and exactly which query term matched which declaration. A wrong route is a wrong declaration or a missing synonym, and it can be fixed in the corpus rather than tuned in a model.

Third, a language model can sit *in front of* this router, rewriting a question into declared vocabulary, but it may not sit *inside* it. The routing decision and its explanation remain a function of the declarations.

Hub corpora are never routing candidates. Mirrors of the same corpus ID are one candidate with several holders, tried local first, then fresh origin holders, then fresh mirrors, then stale ones.

Ambiguity is preserved. When two corpora both plausibly own a question, the result lists both, with their lineage ("fork of C-..."), and returns no page. A node may set precedence to prefer certain corpora, and when it does, the routing note says precedence was used. The prohibition is on silent merging, not on local policy.

8. Provenance That Can Be Checked

A remote answer without remote provenance is a local hallucination with better transportation.

The first draft required that provenance travel with every remote answer. The review pointed out that nothing stopped an intermediary from rewriting it. Provenance that travels but cannot be checked is an assertion by whoever last touched it. This section makes it checkable, with nothing but hashes and one signature.

The chain. Retrieval is static. To answer from corpus C on node B, a reader fetches four things and checks each against the one before:

1. B's description. The signature verifies and B's ID matches its key.
2. C's manifest. Its hash equals the fingerprint B's signed description declares for C.
3. C's catalog. Its hash is listed in the manifest.
4. The page. Its hash is listed in the manifest.

So the provenance of any page reduces to a signature by the node that served it, over a description that names the corpus fingerprint, which names the page hash. Nothing in the chain depends on a hub, a relay, or a cache.

The envelope. Every retrieved page is returned with an envelope:

```
corpus_id, corpus_title, document_id, path, title, version, status,
origin, origin_depth, ratified_by,
content_sha256, manifest_sha256,
served_by { node_id, public_key, description_sha256, endpoint },
route [ {node_id, role: requester | discovery | server} ... ],
retrieved_at, verified, verification[], attestations[]
```

`verify_envelope()` repeats the chain for any third party holding the page, the manifest, and the description, and a node that caches a page keeps all three.

Four separations are enforced by the envelope's structure rather than by convention:

- *Transport is not authorship.* `route` records hops; it has no author field.
- *Discovery is not authority.* A hub appears in `route` with role `discovery`, never `server`.
- *Caching is not authorship.* A caching node adds a signed attestation meaning "this node relayed and stored these bytes; it did not author them".
- *Reasoning is not provenance.* Nothing in the retrieval path generates text.

Verification modes. A node sets `verify` to `require` (reject anything that fails), `warn` (accept, mark `verified: false`), or `off` (testing only). In the reference run, a page altered after its manifest was built is refused under `require` and reported as the owner being unavailable, not silently served.

Unavailable is not impersonated. If the owner is known and unreachable, the answer is "relevant corpus known; node unavailable", with the endpoints tried. A byte-identical mirror of the same corpus ID may serve instead, and the envelope then names both the corpus and the mirror that served it. A cached copy may be mentioned, labelled as a copy with its retrieval date. What may not happen is another corpus's page answering in the owner's place.

9. Supersession Across Corpora

Within one corpus, supersession is the local web's rule: a page is replaced by a new version, the old version moves to `_log/superseded/` as a retired page with its own document ID, the new page records supersedes, and the revision log records why. Nothing is deleted.

Across corpora, the first draft treated `d1` $\rightarrow$ `d2` as a single relation. After differentiation that breaks. Two descendants of one seed can each revise the same ancestral page, differently, and a reader following lineage could mistake a fork's revision for the origin's (a form of FM-N6).

Supersession is scoped to a corpus. A corpus may supersede only its own documents. A qualified reference `C-X:DOC-1` in a `supersedes` field names corpus X, and check N.6 rejects it unless X is this corpus. A page that departs from another corpus's document records that with `diverges-from`. The source's page remains the source's page, and "what the origin says now" and "what this fork says instead" stay distinct.

Supersession propagates without deleting. A node that cached a remote page learns about its supersession by re-reading the owner's catalog. The cached copy is not edited (its envelope is signed as retrieved) and is not removed. A sidecar file records that version n has been superseded remotely by version $n + 1$, and when it was seen.

10. Origin Depth and Epistemic Laundering

A network that machine readers write into faces a failure a human web mostly did not: model output, re-retrieved and re-summarized, returning as if it were source. Call this **epistemic laundering**. Each step looks innocent; the sum is a corpus in which nobody can tell what a human ever asserted.

Every page declares an origin:

- `human`: written by a person, depth 0;
- `machine`: produced by a model or program, depth ≥ 1 ;
- `ratified`: machine-produced, then reviewed and accepted by a named person, depth 0 with a ratification record.

The rules, sharpened from the first draft:

1. **Derived depth is max + 1.** A machine page derived from several sources takes one more than the deepest of them.
2. **Unknown sources are conservative.** A source in another corpus whose depth is not known locally is treated as depth 1, so derivations from it start at depth 2.

3. **Verbatim copying does not reset depth.** A person who pastes machine text into a page has copied it, not written it. Its depth is the depth of what was pasted.
4. **Only ratification resets depth, and ratification must be recorded.** A ratified page names ratified-by and ratified-at, and check N.7 rejects a ratified page without them. An unrecorded ratification would be the laundering channel itself.
5. **Depth is not truth.** It is derivational distance. A human page can be wrong and a depth-3 page right.

The metadata travels; the policy stays local. Each node sets `d_max`, the greatest depth it will serve. A node with `d_max = 0` serves only human and ratified pages. In the reference run a node with `d_max = 1` withholds a depth-2 machine summary that matches a query, returns the best eligible page instead, and reports that one matching page was withheld by policy.

11. Offline, Transport, and Sneakernet

Offline is a state, not a failure. A node disconnected from everything must still validate its corpora, answer local questions, make mirrors and descendants, add spokes, and pack archives. Remote questions must be reported as unavailable, not answered from something else. The experiment measures this as offline retention.

Transport is replaceable. A connector knows one thing: how to read a file at a relative path under an endpoint. It never interprets what it reads. The reference implementation has three: `file://` for a directory on any mounted filesystem; `zip://` for an archive carried by hand; `http://` for a node served by any static web server. Adding a transport is adding one class. Verification does not depend on how bytes arrived.

Sneakernet counts. A ZIP archive on a USB stick is a transport. `mplpb-net pack` writes a corpus or a whole node into one, excluding keys; `unpack` refuses entries that would escape the destination. In the experiment, the new-generation node is seeded from a ZIP of a surviving copy, not over any network.

12. Local Policy

Configuration is local policy, stored in `config.json` on the node, never in a corpus. Two nodes carrying the same corpus may serve it under different rules. Following the toggle policy of the Smart Local front end, every key changes observable behaviour and each is exercised by a test; a key that did nothing would be a claim the code does not keep.

Key	Default	Effect
<code>d_max</code>	1	greatest origin depth served
<code>ownership_margin</code>	1.5	factor by which the leader must beat the runner-up
<code>min_scope_score</code>	2.0	below this, a corpus owns nothing
<code>precedence</code>	[]	corpus IDs that win ambiguities; always reported
<code>offline</code>	false	refuse network connectors; file and archive still work
<code>allow_connectors</code>	file, zip, http	which transports may be used at all

Key	Default	Effect
timeout_seconds	5.0	HTTP timeout
verify	require	require / warn / off
stale_after_days	30	when a description is reported stale
max_hops	2	hub-to-hub steps followed in discovery
cache_remote	true	keep retrieved pages with envelopes
include_retired	false	let retrieval return retired pages, marked

13. The Death Test and the 2010 Computer Test

The death test. Assume the author is gone. No messages, no clarifications, no original machine. A stranger holding one copy must be able to recover what the corpus is, what it claims to own, which pages are current, what each page superseded, where the corpus came from, and how to make another copy that validates. Recovery fails if any of these needs the author.

The 2010 computer test. Assume an old machine: a browser, a file manager, a hash tool, perhaps an old Python. The corpus must be readable and verifiable there.

The first draft leaned on Python tooling for validation. HTML is plausibly durable across decades; a particular interpreter version is less so. The revision is that **the validation rules travel with the seed in prose**. `spec/validation.html` states each of the sixteen checks in plain language, is generated from the same table the validator enforces, and a unit test fails if any check the code performs is missing from the prose. Integrity verification needs only SHA-256 and the instruction, written in `spec/seed.html`, that the fingerprint is the hash of the file list as compact sorted JSON. A reader who has never seen the software can re-implement the checks. Whether readers actually can is falsifier F11.

14. Reference Implementation

The repository `networked-mplpb` is laid out like its two predecessors (the Smart Local front end and the Local Mirror back end): a package, `Tests/`, `Tools/`, `Docs/`, an example corpus, a dual licence. It imports nothing outside the Python standard library, which the experiment checks by parsing every module's imports.

```

mplpb_net/
  corpus.py      the seed: identity, lineage, manifest, catalog, spokes, pages, supersession
  validate.py    16 checks: 11.1–11.8 local, N.1–N.8 network
  replicate.py   mirror / descendant / fork / partial; pack / unpack
  node.py       Ed25519 identity, signed descriptions, learning, staleness
  hub.py        a corpus of descriptions; register, refresh, rebuild; never answers
  router.py     scope routing with vocabulary, margin, precedence, ambiguity
  retrieve.py    static retrieval, envelopes, verification, cache, supersession refresh
  connectors/    file, zip, http
  config.py     twelve policy keys
  server.py     optional static server; never serves local/, known/, cache/, config
  ed25519.py    RFC 8032 in pure Python
  experiment.py Conditions A–H
  scale.py      FM-N15 probe

```

Seed/ the reference seed
 Tests/ 61 unittest cases
 Docs/ this paper (md, txt, pdf), lineage, blind-recovery protocol, results

Corpora are compatible with the local web in both directions: pages use the same metadata fields, updated format, supersession convention, and insertion marker, and a seed produced here passes the Smart Local front end's own eight checks and answers its ask command.

The sixteen checks.

Check	Name	What it rejects
11.1	Link validity	broken or absolute internal links
11.2	Root reachability	orphan pages
11.3	Required metadata	missing fields or upward links
11.4	Unique identity	two current pages with one ID
11.5	Supersession consistency	retired pages in the wrong place, dangling supersedes
11.6	Boundary safety	links escaping the corpus
11.7	Index consistency	pages missing from their Sub-Index
11.8	Timestamp format	unparseable updated values
N.1	Corpus identity	missing identity, bad relation, stale twin
N.2	Manifest integrity	any changed, missing, or unlisted file
N.3	Lineage chain	a broken hash chain
N.4	Declared scope	empty scope, terms outside the vocabulary
N.5	No machine identity inside	keys or node files in a corpus
N.6	Scoped supersession	superseding another corpus's document
N.7	Origin depth	wrong depth, understated derivation, unrecorded ratification
N.8	Seed completeness	anything a stranger needs to make the next copy

A minimal session:

```
mplpb-net node init A --name alpha
mplpb-net node seed A --title "Seed" --scope "Corpus rules" --terms corpus,validation
mplpb-net node init B --name beta
mplpb-net node carry B A/corpora/C-... --relation descendant \
  --title "Kiln Notes" --scope "Kiln firing and glazes" --terms kiln,glaze,firing
mplpb-net spoke B/corpora/C-... kiln --title Kiln --scope "Firing schedules"
mplpb-net add B/corpora/C-... kiln --title "Bisque firing" --scope "Bisque schedule" --text "..."
mplpb-net node describe B
mplpb-net hub init H --name hub-1
mplpb-net hub register H file:///.../B
mplpb-net node learn A file:///.../H
mplpb-net ask A "bisque firing schedule for a kiln"
```


15. The Seed-to-Network Experiment

15.1 Conditions

The experiment is a sequence of growths and removals, runnable with `mplpb-net experiment [--transport file|http]`.

Condition	What happens
A. Seed	Node A creates the seed, adds a spoke and a page, supersedes the page.
B. Replication	B, C, D each take a mirror. Each must validate and match the seed fingerprint.
C. Differentiation	B, C, D each grow a descendant with its own domain (kilns, bees, bicycles). D also carries a <i>fork</i> of C (rooftop bees), creating a real scope collision, and a machine-written page at depth 2.
D. Discovery	A hub is created; all four nodes register; all four learn the hub.
E. Cross-node retrieval	A asks twelve graded queries. B then supersedes a page A has cached; A refreshes. B goes offline.
F. Hub loss	The hub is stopped and deleted. A asks every routable query again.
G. Origin loss	Node A is stopped and deleted.
H. New generation	A copy of the seed is carried from C to a new node E as a ZIP. A new hub is rebuilt from descriptions surviving on B, C, D. E joins and asks.

The query set is graded as the review recommended: clear local, clear remote, synonym-only (vocabulary mismatch within one lineage), true collision between a corpus and its fork, partial overlap resolved by one distinguishing term, and absent domains.

15.2 Metrics

Metric	Definition
Recovery rate R	fraction of ten properties recovered from E's copy with no contact with A
Routing accuracy A_r	routable queries sent to the correct corpus
Ambiguity preservation A_p	true collisions reported as ambiguous, with no page returned
Provenance retention P_r	remote answers whose envelope re-verifies from cached artifacts and names the true source
Offline retention O_r	local capabilities still working when fully disconnected
Replication integrity I_r	copies that validate and, for mirrors, match the fingerprint
Origin independence I_o	surviving copies intact after the origin is deleted
Infrastructure burden	non-stdlib imports, required services, administrative actions, time, memory, storage

15.3 Results of the reference run

Both transports were run on one machine (Python 3.12, Linux). Every node in the HTTP run was served by the standard-library server on localhost.

Metric	File	HTTP
Routing accuracy A_r	1.000	1.000
Ambiguity preservation A_p	1.000	1.000
Absent domains reported	1.000	1.000
Provenance retention P_r	1.000	1.000
Depth policy respected; laundering probe withheld	yes; yes	yes; yes
Supersession propagated; cached history kept	yes; yes	yes; yes
Offline retention O_r ; remote reported unavailable	1.000; yes	1.000; yes
Answered after hub loss	1.000	1.000
Origin independence I_o	1.000	1.000
Recovery rate R (Condition H)	1.000	1.000
New-generation routing	1.000	1.000
Replication integrity I_r	1.000	1.000
Non-stdlib imports	0	0
Required persistent services	0	optional static server per node
Administrative actions	21	21
Wall time; peak memory	2.4 s; 27 MB	2.9 s; 30 MB

The ten recovered properties in Condition H were: corpus ID, fingerprint, readable scope, boot block, validation rules in prose, current versus retired pages, a traceable supersession chain, a verifying lineage chain, the instantiation guide, and the ability to seed a validating next generation.

Two results are worth more than the ones. Hub loss cost nothing in Condition F because every node had already cached the signed descriptions it learned; the hub was needed to *find* peers, not to *reach* them. And the new hub in Condition H was rebuilt entirely from descriptions held by the survivors, which verified because they were signed by their nodes, not by the lost hub.

15.4 Scale probe

`mplpb-net scale` registers N synthetic nodes with one hub and measures discovery cost.

Nodes	Register, per node	Registry size	Learn all	Route, per query	Routing correct
10	33 ms	15 KB	0.08 s	1.0 ms	10/10
40	48 ms	58 KB	0.30 s	3.6 ms	10/10
160	120 ms	234 KB	1.5 s	14.0 ms	10/10

Registry size and routing cost grow linearly, as expected for a flat list scanned per query. Per-node registration cost grows about $3.6\times$ for a $16\times$ increase in nodes, because the

reference hub rehashes its whole corpus on every registration, making total registration cost roughly quadratic. This is an instance of scale inversion (FM-N15) in the reference implementation, not necessarily in the architecture: incremental manifests or batched registration would remove it. It is reported because the prediction was that something would bend, and this is where.

16. What the Reference Run Does Not Establish

A clean sweep on an author-built network is the result most likely to mislead, so this section is as long as it needs to be.

It is a mechanical rehearsal. The corpora, queries, expected answers, and code share one author. Every node runs on one machine. The run shows that the implementation does what the specification says. It does not show that the specification is right.

The routing numbers measure the declarations. Twelve queries were written by someone who knew the scopes. Ambiguity preservation is the metric most likely to look good on a small hand-built network and degrade in the wild, where collisions are partial, vocabularies drift, and nobody wrote a synonym for the word a stranger uses.

Recovery was tested by the software, not by a stranger. Condition H checks that the properties are present and machine-verifiable. It does not check that a person who has never seen the project can find and use them. That is F1, and it is the central claim.

There was no baseline. Nothing here shows that declared-scope routing beats flat retrieval over the union of all corpora.

There were no adversaries. Signatures make forgery detectable; they do not stop a node from signing a false scope, a hub from omitting entries, or anyone from claiming a corpus ID they did not create (FM-N16).

Two protocols address the first four points and are specified in Docs/PROTOCOL.md.

Blind stranger recovery. Recruit participants who have never seen the project, the author, or anyone briefed by the author. Give each a ZIP of one descendant corpus and nothing else: no README from the repository, no software. Ask them, in writing, to answer ten fixed questions (what corpus is this; what does it claim to own; which pages are current; what did page X replace; where was it seeded from; is this copy intact; make a copy that adds one page and remains valid by the corpus's own rules; and so on). Grade answers against a key written before recruitment, by a grader who does not know which participant received which corpus. Report R per participant and the questions that failed. A second arm gives participants only spec/validation.html and asks them to implement the checks in any language and run them on ten corpora, three of them deliberately broken; agreement with the reference validator tests F11.

Ablation. Same corpora, same graded query set enlarged to at least one hundred queries written by people other than the corpus authors, three conditions: (a) declared-scope routing as specified; (b) flat BM25 over all pages of all corpora with no scope information; (c) (b) plus a rule that returns "ambiguous" when the top two results come from different corpora within a score margin. Measure A_r , A_p , and the rate at which each condition returns a page

from the wrong corpus. If (b) or (c) matches (a), declared scope is decoration and F3 holds.

17. Failure Modes

ID	Name	Description	Mitigation in this design
FM-N1	Scope inflation	a corpus declares more than it can answer	inspectable routing; margin; vocabulary review
FM-N2	False merger	answers from several corpora blended into one	ambiguity outcome; no merge path in code
FM-N3	Provenance stripping	a remote page loses its source in transit	signed chain; envelope; third-party verification
FM-N4	Hub capture	a hub becomes the answerer or authority	hubs are corpora of descriptions; no query-to-content function
FM-N5	Stale discovery	dead nodes treated as live	last_seen; stale_after_days; stale holders tried last
FM-N6	Lineage confusion	a fork or descendant read as its origin	declared relation; seeded-from; scoped supersession
FM-N7	Epistemic laundering	machine text re-enters as source	origin depth; max+1; recorded ratification; d_max
FM-N8	Impersonation of the unavailable	another corpus answers for an unreachable owner	unavailable outcome; mirrors only by same corpus ID
FM-N9	Silent overwrite	revision destroys history	supersession to _log/superseded/; sidecars not edits
FM-N10	Seed incompleteness	a copy lacks what a stranger needs	check N.8; spec/seed.html; prose rules
FM-N11	Network dependence creep	local functions start needing the network	offline retention measured; connectors optional
FM-N12	Replication corruption	a copy differs undetectably	manifest; fingerprint; refuse to copy unverified sources
FM-N13	Hub poisoning	a hub serves forged or misleading descriptions	per-description signatures; multiple hubs
FM-N14	Identity collision	two machines claim one identity	ID derived from key; keys never inside corpora
FM-N15	Scale inversion	structure becomes the bottleneck	measured by the scale probe; observed in hub registration
FM-N16	Corpus-ID claim	anyone can claim an existing corpus ID	open; see Section 19
FM-N17	Vocabulary drift	descendants' synonyms diverge until routing degrades	vocabulary inherited from seed; open beyond one lineage

FM-N16 and FM-N17 were found while implementing, not while writing the first draft.

18. Falsifiers

The architecture is wrong, or wrong in part, if any of the following is observed under the protocols above.

- **F1.** Strangers given one copy cannot recover what the corpus is, what it owns, and which pages are current, at a rate a reasonable reviewer would accept (blind protocol, R).
- **F2.** Copies that pass validation on one machine fail to be interpretable or valid on another.
- **F3.** Declared-scope routing performs no better than flat retrieval (ablation).
- **F4.** True scope collisions are routed to a single corpus or merged, rather than reported, at a material rate on queries written by outsiders.
- **F5.** Provenance does not survive routing: envelopes fail third-party verification or name the wrong source.
- **F6.** Loss of a hub disables anything beyond finding new peers.
- **F7.** Loss of the origin removes the ability to recover or re-seed the corpus.
- **F8.** Infrastructure burden grows with the network: nodes need services, dependencies, or administration that a single local corpus did not.
- **F9.** Origin depth fails to stop laundering: machine-derived material reaches a $d_{\max} = 0$ node as depth 0 without a ratification record.
- **F10.** Discovery cost grows so fast that the network cannot exceed a size useful in practice, and no change within the design fixes it.
- **F11.** Independent readers given only the prose rules cannot re-implement validation that agrees with the reference validator.

The reference run bears on F2, F5, F6, F7, F8, and F9 mechanically, and on F10 only for the reference hub. It does not bear on F1, F3, F4 as stated, or F11.

19. Limitations and Open Problems

Corpus IDs are claims. Node identity is bound to a key; corpus identity is not. A node can declare that it carries corpus C-X with any content, and a reader will see that its fingerprint differs from other holders' but cannot tell which is the real C-X. Mirrors of an origin that has since changed show the same symptom innocently. A corpus key that signs manifests would fix attribution at the cost of key management across generations, which conflicts with the death test: a corpus whose author is gone can no longer sign. The likely answer is that corpus identity should be the pair (ID, lineage-root hash), with forks distinguished by lineage, but it is not specified here.

Declared scope can lie. Signatures prove who declared a scope, not that the declaration is honest. Scope inflation by a signed node is visible only through bad answers. Reputation is deliberately out of scope, because it would make hubs into authorities.

Vocabulary only helps within a lineage. Unrelated seeds still describe overlapping domains in different words. A cross-lineage vocabulary is itself a shared resource with the same governance problems as a hub.

The reference hub does not scale well. Registration rehashes the hub corpus; routing scans every known description. Both are fixable with incremental manifests and an index over scope terms, neither is done.

The signature code is not hardened. It is correct against the RFC vectors and adequate for tamper evidence. It is not constant-time.

Retrieval inside a corpus is crude. A small TF-IDF over the catalog chooses one page. The architecture does not depend on it, and a better retriever can replace it, but the reported numbers do.

One machine. Clock skew, partial network failure, and hostile endpoints have not been exercised.

20. Conclusion

The first draft argued that a knowledge network could grow from seeds with almost no persistent infrastructure. The revision makes that argument checkable. A seed now carries its identity, integrity, lineage, vocabulary, and validation rules inside itself; a node's identity is its key; a hub is a corpus that can be rebuilt from what survives; routing reports ambiguity instead of hiding it; provenance reduces to a signature a stranger can verify; and machine-derived material carries its distance from a human source, with ratification as the only way back and a record required for it.

A standard-library implementation shows that all of this runs, on file and HTTP transport, through the loss of the origin and the hub, into a new generation seeded by hand. That is the easy half. The hard half is a stranger with a ZIP file and no one to ask, and a hundred questions written by people who never saw the scopes. The protocols for both are written. The claim stands or falls on them.

References

These references were compiled from memory, without access to search or a bibliographic database. Titles, venues, and years should be checked against the original sources before this paper is submitted or cited.

- Alexander, K., Cyganiak, R., Hausenblas, M., Zhao, J. *Describing Linked Datasets with the VOID Vocabulary*. W3C Interest Group Note, 2011.
- Benet, J. *IPFS: Content Addressed, Versioned, P2P File System*. arXiv:1407.3561, 2014.
- Callan, J. P., Lu, Z., Croft, W. B. Searching distributed collections with inference networks. *Proc. ACM SIGIR*, 1995.
- CCSDS. *Reference Model for an Open Archival Information System (OAIS)*. CCSDS 650.0-M-2, 2012; ISO 14721:2012.
- Gravano, L., García-Molina, H., Tomasic, A. GLOSS: text-source discovery over the Internet. *ACM Transactions on Database Systems*, 1999.
- Gravano, L., Chang, C.-C. K., García-Molina, H., Paepcke, A. STARTS: Stanford proposal for Internet meta-searching. *Proc. ACM SIGMOD*, 1997.

- Groth, P., Gibson, A., Velterop, J. The anatomy of a nanopublication. *Information Services & Use*, 2010.
- Jacobson, V., et al. Networking named content. *Proc. ACM CoNEXT*, 2009.
- Josefsson, S., Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.
- Kuhn, T., Dumontier, M. Trusty URIs: verifiable, immutable, and permanent digital artifacts for linked data. *Proc. ESWC*, 2014.
- Kunze, J., Littman, J., Madden, E., Scancella, J., Adams, C. *The BagIt File Packaging Format (V1.0)*. RFC 8493, 2018.
- Lagoze, C., Van de Sompel, H. The Open Archives Initiative: building a low-barrier interoperability framework. *Proc. ACM/IEEE JCDL*, 2001.
- Lewis, P., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in NeurIPS*, 2020.
- Maniatis, P., Roussopoulos, M., Giuli, T. J., Rosenthal, D. S. H., Baker, M. The LOCKSS peer-to-peer digital preservation system. *ACM Transactions on Computer Systems*, 2005.
- Merkle, R. C. A digital signature based on a conventional encryption function. *Proc. CRYPTO '87*, 1987.
- Moreau, L., Missier, P. (eds.) *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013.
- Soiland-Reyes, S., et al. Packaging research artefacts with RO-Crate. *Data Science*, 2022.
- Tarr, D., Lavoie, E., Meyer, A., Tschudin, C. Secure Scuttlebutt: an identity-centric protocol for subjective and decentralized applications. *Proc. ACM ICN*, 2019.
- Van de Sompel, H., Nelson, M., Sanderson, R. *HTTP Framework for Time-Based Access to Resource States — Memento*. RFC 7089, 2013.
- Anthropic. *Model Context Protocol* specification, 2024.
- Kiwix and openZIM project documentation (software; no archival paper).
- Git (software), L. Torvalds and contributors, 2005–.

Related work by the author. McPhetridge, M. D. *Smart Local MPLPB: A Retrieval-Bounded Front End for a Local Web*, 2026 (github.com/mitchell-d00/SMART-LOCAL-MPLPB-A-Retrieval-Bounded-Front-End-for-a-Local-Web). *The Local Mirror: MPLPB as a Self-Contained Offline Site*, 2026 (github.com/mitchell-d00/-The-Local-Mirror-MPLPB-as-a-Self-Contained-Offline-Site). *Continuity Without Memory*, 2026 (Google Play Books, id zhUAEgAAQBAJ).

Appendix A. An envelope (development session; hashes elided)

```
{
  "corpus_id": "C-9BDBB6132CCA",
  "corpus_title": "Kiln Notes",
  "document_id": "KILN-001",
  "path": "kiln/bisque_firing.html",
  "version": 1,
  "status": "current",
  "origin": "human",
  "origin_depth": 0,
  "content_sha256": "...",
  "manifest_sha256": "...",
  "served_by": {
    "node_id": "N-21284C75FCE1DE36",
    "public_key": "...",
    "description_sha256": "...",
    "endpoint": "http://127.0.0.1:40507/"
  },
  "route": [
    {"node_id": "N-DAF79EA363CDB2D3", "role": "requester"},
    {"node_id": "N-7FC669B5D1DA93FA", "role": "discovery"},
    {"node_id": "N-21284C75FCE1DE36", "role": "server"}
  ],
  "verified": true,
  "attestations": [
    {"node_id": "N-DAF79EA363CDB2D3", "role": "cache",
     "meaning": "this node relayed and stored these bytes; it did not author them",
     "signature": "..."}
  ]
}
```

Appendix B. Revision notes

Changes from the first draft, each prompted by review:

1. Scope is now declared against a vocabulary carried in the seed, and routing is inspectable term by term (Section 7).
2. Provenance is verifiable: content hashes, a corpus fingerprint, and Ed25519-signed node descriptions, implemented without leaving the standard library (Sections 4, 8).
3. Supersession is scoped to a corpus; departures from another corpus use diverges-from (Section 9).
4. Origin-depth rules cover multiple sources ($\text{max} + 1$), unknown sources, verbatim copying, and require ratification records (Section 10).
5. The field is surveyed and the contribution positioned against it early (Sections 1.2, 2).
6. Validation rules travel in prose inside the seed, and a test keeps prose and code in step (Section 13).
7. The experiment builds graded scope collisions into the query set, and the blind-recovery and ablation protocols are specified for participants who have never seen the corpus (Sections 15, 16).